Q27 — Comprehension check

Deep Learning Systems 2026 — AI Internals Module

Comprehension Question

A student writes:

> *"Interpretability: how can inspecting weights reveal a model's tendencies? And do models revert to their pre-training traits after deployment?"*

Address both parts. First, explain how weight- and activation-level inspection can expose a model's learned structure and behavioral dispositions, naming at least two concrete interpretability techniques and stating clearly what they do and do not establish. Second, assess the claim that a deployed model "reverts" to pre-training traits: distinguish between (a) persistent parameters fixed at deployment, (b) in-context adaptation that does not alter weights, and (c) the re-emergence of pretrained behaviors when fine-tuning constraints are weak. Be explicit about which framings are supported by current evidence and which are misconceptions.

Answer **Part 1 — Inspecting internal structure.** A trained network stores its dispositions in parameters, but raw weights are rarely directly legible; interpretability instead reconstructs human-understandable features and circuits from weights and activations. Several complementary techniques exist. (i) *Feature visualization and attribution* characterize what inputs maximally excite a unit or channel, giving evidence about what a neuron responds to [1]. (ii) *Circuit analysis* traces how small subgraphs of weights compose simple features into higher-level computations, with the "curve detector" and related circuits in vision models as canonical worked examples [2]. (iii) *Probing classifiers* train a simple readout on frozen activations to test whether a property (e.g., syntax, sentiment) is linearly recoverable; crucially, a successful probe shows information is *present*, not that the model *uses* it, and probe accuracy must be calibrated against controls [3]. (iv) *Sparse dictionary methods* decompose polysemantic activations into more monosemantic features, addressing superposition where one neuron encodes many concepts [4]. Taken together, these tools can surface tendencies—such as reliance on spurious shortcut features—but inferences about "what the model will do" are correlational unless validated by causal interventions (ablating or patching the identified components and observing behavioral change). Weight inspection reveals *capacities and mechanisms*, not guaranteed behavior across all deployment inputs.

Part 2 — "Reverting" after deployment. The intuition conflates three distinct phenomena. (a) *Fixed parameters.* For a standard deployed model, weights are frozen at inference time; the model does not spontaneously drift back toward a pre-training checkpoint, because no gradient updates occur. There is no mechanism for such reversion. (b) *In-context adaptation.* Large models adjust behavior from prompt context without any weight change, an effect sometimes framed as implicit optimization over the context [5]. This is transient and input-local: changing the prompt changes behavior, but the parameters—and thus the underlying dispositions—are unchanged. Describing this as "reverting" is a misconception; it is conditioning, not relapse. (c) *Re-emergence of pretrained*

behavior under weak fine-tuning. The kernel of truth is that alignment or safety fine-tuning is often a thin, shallow modification over a vast pretrained base; adversarial prompts, distribution shift, or further fine-tuning can elicit pretrained behaviors that the fine-tuning only superficially suppressed [6]. This is best understood not as autonomous reversion over time but as the fine-tuned policy failing to cover inputs far from its training distribution, exposing the still-present pretrained capacities documented by the interpretability tools above. **Summary:** inspecting internals reveals mechanisms and latent tendencies but requires causal validation to predict behavior; deployed weights do not drift back to pre-training on their own, yet pretrained dispositions remain latent and can resurface when the fine-tuning layer is shallow or the input is out-of-distribution.

References

[1] C. Olah, A. Mordvintsev, and L. Schubert. Feature visualization. *Distill*, 2017. https://distill.pub/2017/feature-visualization/.

[2] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter. Zoom in: An introduction to circuits. *Distill*, 2020. https://distill.pub/2020/circuits/zoom-in/.

[3] J. Hewitt and P. Liang. Designing and interpreting probes with control tasks. In *Proceedings of EMNLP-IJCNLP*, 2019.

[4] N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, S. Kravec, Z. Hatfield-Dodds, R. Lasenby, D. Drain, C. Chen, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/toy_model/index.html.

[5] S. Garg, D. Tsipras, P. Liang, and G. Valiant. What can transformers learn in-context? A case study of simple function classes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[6] A. Wei, N. Haghtalab, and J. Steinhardt. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.